

CAPSTONE PROJECT REPORT

CLOUD SECURITY BEST PRACTICES

GROUP 6

Lead Engineer: SIXTUS OKORO

July 17, 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. AWS Services Used
5. Tools Used
6. Implementation Steps
 - 6.1 Security Groups Review
 - 6.2 Identity and Access Management (IAM)
 - 6.3 Key Pair Management
 - 6.4 Amazon S3 Permissions
 - 6.5 GitHub Repository Setup
7. Screenshots and Documentation
8. Challenges
9. Lessons Learned
10. Conclusion
11. References

1. Introduction

Cloud computing has revolutionized how organizations deploy and manage infrastructure. However, the migration to platforms like Amazon Web Services (AWS) introduces a paradigm shift in security responsibilities. AWS operates under a Shared Responsibility Model, meaning AWS is responsible for the security 'of' the cloud (hardware, global infrastructure), while the customer is responsible for security 'in' the cloud (customer data, IAM, operating systems, and firewall configurations).

This capstone project, executed by Group 6, outlines the core principles of securing a small AWS environment. We focus on critical areas that are statistically prone to misconfiguration: Security Groups, Identity and Access Management (IAM), SSH Key Pair management, and Simple Storage Service (S3) bucket permissions. Through a structured review and remediation strategy, this report provides a blueprint for establishing a robust security posture for cloud-native applications.

2. Problem Statement

Small AWS environments, particularly those built during rapid development cycles or proof-of-concept phases, often suffer from systematic security misconfigurations. Common vulnerabilities include overly permissive Security Groups (e.g., port 22 or 3389 open to the global internet (0.0.0.0/0)), root account usage without Multi-Factor Authentication (MFA), hardcoded IAM credentials,

and publicly readable/writable S3 buckets.

These misconfigurations expose the infrastructure to automated botnets, ransomware operators, and data exfiltration threats. The challenge addressed in this project is to systematically review a theoretically vulnerable AWS architecture, identify the weak points across compute, storage, and identity layers, and apply industry-standard security best practices to harden the environment against external and internal threats.

3. Objectives

The primary objectives of this capstone project are to:

- Review Security Groups: Analyze and restrict inbound and outbound traffic to enforce the principle of least privilege.
- Review IAM Concepts: Establish a theoretical and practical framework for identity management, focusing on roles, policies, and secure access mechanisms.
- Discuss Key Pair Management: Evaluate secure practices for generating, storing, and utilizing SSH keys for instance access.
- Review S3 Permissions: Harden object storage by restricting public access, enforcing encryption, and utilizing bucket policies.
- Store the Security Report in GitHub: Demonstrate version control and collaborative documentation practices.

- Present Security Recommendations: Deliver a comprehensive live demonstration showcasing the secure environment.

4. AWS Services Used

The security review heavily engaged the following core AWS services:

- Amazon Elastic Compute Cloud (EC2): Used as the primary compute environment requiring Key Pair and Security Group hardening.
- Amazon Virtual Private Cloud (VPC): The foundational networking layer housing the Security Groups.
- AWS Identity and Access Management (IAM): Utilized for defining users, groups, roles, and granular permission boundaries.
- Amazon Simple Storage Service (S3): Object storage utilized for securing application data and logs, heavily dependent on bucket policies and access block settings.

5. Tools Used

- Git & GitHub: For version control, report storage, and collaborative code review.
- Docker: Utilized during the live demonstration to containerize application workloads securely.
- AWS Command Line Interface (CLI): Used for programmatic interaction and auditing of AWS resources.
- Draw.io / Lucidchart: Utilized to design the target Architecture Diagram.

- Markdown: Used to create the README.md documentation inside the GitHub repository.

6. Implementation Steps

6.1 Security Groups Review

Security Groups act as stateful virtual firewalls at the Elastic Network Interface (ENI) level. In an unoptimized environment, it is common to find the 'default' security group loosely configured, or custom security groups allowing global access. Our remediation approach included:

1. Principle of Least Privilege: We audited all inbound rules. We explicitly removed any rule allowing SSH (Port 22) from 0.0.0.0/0. Instead, SSH access was restricted to the specific static IP address of the administrative bastion host or corporate VPN gateway.
2. Separation of Tiers: Web server security groups were configured to accept HTTP/HTTPS (Ports 80/443) only from the Application Load Balancer's security group, not the open internet. Database security groups (e.g., Port 3306 for MySQL) were strictly locked down to only accept connections from the Web Server security group.
3. Outbound Rules: Default outbound rules (Allow All) were evaluated. While often left open, best practice dictates restricting outbound traffic to required services only (e.g., allowing port 443 for OS updates) to prevent reverse-shell backdoors or data exfiltration.

6.2 Identity and Access Management (IAM)

Identity is the new security perimeter in cloud computing. Our review of IAM centered around replacing long-term credentials with short-term, role-based access:

1. Root User Protection: The root account was secured with a hardware MFA token. All access keys associated with the root user were deleted. The root account is exclusively reserved for billing and account-level emergency tasks.
2. IAM Users and Groups: Individual IAM users were created for administrators, strictly mapped to IAM Groups (e.g., 'CloudAdmins', 'Developers'). Permissions are assigned to the group, never directly to the user.
3. IAM Roles for EC2: We identified instances using hardcoded AWS credentials in their configuration files. This anti-pattern was resolved by creating an IAM Role with an attached policy granting specific S3 access, and attaching this role via an Instance Profile to the EC2 instance.
4. Password Policies: We enforced a strict password policy requiring a minimum of 14 characters, complexity requirements, and a 90-day expiration.

6.3 Key Pair Management

Accessing EC2 instances securely requires stringent cryptographic key management. The following protocols were established:

1. Key Generation and Storage: Keys were generated using ED25519 (preferred over RSA for performance and security). Private keys (.pem/.ppk) were never

stored in source code or shared among engineers.

2. File Permissions: Linux file permissions for the private key were strictly enforced via 'chmod 400 key.pem' to ensure read-only access for the file owner.

3. Modern Alternatives discussed: As part of our forward-looking recommendations, we advocated for transitioning away from traditional SSH keys entirely. Utilizing AWS Systems Manager (SSM) Session Manager allows administrators to establish secure, audited terminal sessions directly from the AWS Console or CLI without requiring open inbound ports or SSH keys.

6.4 Amazon S3 Permissions

Data breaches involving S3 are almost exclusively due to customer misconfiguration. To secure our object storage environment, we executed the following checks:

1. Block Public Access (BPA): The S3 Block Public Access feature was enabled at the account level to override any bucket-level or object-level ACLs that might accidentally expose data.

2. Bucket Policies: We implemented JSON-based bucket policies restricting access to the specific IAM Role assumed by our EC2 instances. We included a 'Deny' statement for unencrypted HTTP access (aws:SecureTransport: false).

3. Encryption and Versioning: S3 Default Encryption was enabled using Amazon S3-managed keys (SSE-S3). Object Versioning was turned on to provide a recovery mechanism against accidental deletion or ransomware overwrites.

6.5 GitHub Repository Setup

To ensure reproducibility and transparency, all project artifacts were checked into a central GitHub repository.

1. A '.gitignore' file was strictly configured to prevent the accidental commit of sensitive files, such as AWS credentials, .pem keys, or local environment variables (.env).
2. The repository includes a comprehensive 'README.md' detailing the environment setup, architecture, and deployment instructions.
3. This security report and all generated network diagrams were organized into a '/docs' directory for easy review.

7. Screenshots and Documentation

In a production run of this report, this section contains visual evidence of configuration. For this written submission, please refer to the project GitHub repository for high-resolution images.

- Figure 1: EC2 Security Group inbound rules restricted to Administrator IP.
- Figure 2: IAM Password Policy enforcement dashboard.
- Figure 3: S3 Bucket 'Block all public access' confirmation.
- Figure 4: IAM Role attachment to the EC2 target instance.

8. Challenges

Throughout the implementation of these security practices, Group 6 encountered several challenges:

1. **Balancing Security and Usability:** Initially, restricting the Security Groups was done too aggressively, which broke the application's ability to communicate with the database tier. It required careful troubleshooting of VPC Flow Logs to identify the exact ports needed.
2. **IAM Policy Complexity:** Authoring least-privilege IAM policies manually is error-prone. We encountered 'Access Denied' errors when our EC2 instances attempted to access S3 because our policy lacked the 's3:ListBucket' permission, highlighting the granularity of AWS permissions.
3. **Key Pair Loss:** Simulated scenarios of a lost SSH key required us to document recovery procedures, such as detaching the EBS volume and mounting it to a recovery instance to inject a new public key.

9. Lessons Learned

This capstone project reinforced several critical cloud engineering philosophies:

- **Security is Continuous:** A secure posture is not a one-time setup. It requires constant auditing, preferably automated via tools like AWS Security Hub or AWS Config.
- **Least Privilege is Non-Negotiable:** Starting with zero permissions and explicitly

granting only what is required minimizes the blast radius of any potential credential compromise.

- Automation over Manual Config: While we configured these settings manually via the Management Console for learning purposes, doing this at scale requires Infrastructure as Code (IaC) tools like Terraform or AWS CloudFormation to avoid human error.

10. Conclusion

Securing an AWS environment requires a comprehensive understanding of how network controls, identity management, and storage policies interact. By reviewing and strictly configuring Security Groups, migrating to IAM role-based access, securely managing key pairs, and locking down S3 buckets, we have successfully transformed a vulnerable baseline architecture into a hardened, production-ready environment. The methodologies applied in this capstone reflect industry best practices and provide a scalable foundation for future cloud deployments.

11. References

1. Amazon Web Services. (2024). Shared Responsibility Model. AWS Security Documentation.
2. Amazon Web Services. (2024). IAM Best Practices. AWS Identity and Access Management User Guide.
3. Amazon Web Services. (2024). Security best practices for Amazon S3.

Amazon S3 User Guide.

4. Center for Internet Security (CIS). (2024). CIS Amazon Web Services Foundations Benchmark.